

ASSIGNMENT 03 (QUIZ 5A)

Name: _____

Roll No: _____

COAL – FALL 2024

Section: _____

Question 01**(5 Marks)**

Dry run the following code and update the memory at the end of execution.

```
.data
array DWORD 0648B2165h, 08C943A29h, 06DFA4B86h
```

```
.code
    mov bl, 4
    mov esi, OFFSET array
    mov ecx, (LENGTHOF array) - 1
```

```
L1:
    push ecx
    mov eax, [esi + TYPE DWORD]
    mov cl, bl
    shrd [esi],eax,cl

    add esi, TYPE DWORD
    pop ecx
    Loop L1
```

```
shr DWORD PTR [esi],4
```

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
4000	16	B2	48	96	A2	43	C9	68	B8	A4	Df	06				

Question 02**(5 Marks)**

Fill in the table for the following instructions and make sure to perform the calculations. Note: The hexadecimal representation of '0' is 30h.

<pre>.code main proc mov ax, 0 mov al, '9' sub al, '1' add al, '7' aaa or ax, 3030h</pre>	<table><tr><td>ax</td><td>0000</td></tr><tr><td>al</td><td>39</td></tr><tr><td>al</td><td>08</td></tr><tr><td>al</td><td>3f</td></tr><tr><td>ax</td><td>3135</td></tr></table>	ax	0000	al	39	al	08	al	3f	ax	3135	<u>CALCULATION</u>
ax	0000											
al	39											
al	08											
al	3f											
ax	3135											
<pre>.code main proc mov ax, 0 mov al, '6' sub al, '9' aas or ax, 3030h</pre>	<table><tr><td>ax</td><td>0000</td></tr><tr><td>al</td><td>36</td></tr><tr><td>al</td><td>fd</td></tr><tr><td>al</td><td>07</td></tr><tr><td>ax</td><td>Ff 37</td></tr></table>	ax	0000	al	36	al	fd	al	07	ax	Ff 37	<u>CALCULATION</u>
ax	0000											
al	36											
al	fd											
al	07											
ax	Ff 37											

Question 03**(5 Marks)**

Execute the following code and fill the table accordingly.

```
.data
```

```
dwordVal SDWORD 50000
```

```
.code
```

```
mov eax, dwordVal
```

```
cdq
```

```
mov ebx, -256
```

```
idiv ebx
```

	Answers	Registers (EAX/EBX/ECX/EDX)
Quotient	ffffff3d / -195 in dec	EAX
Remainder	00000050 / 80 in dec	EDX

```
.data
```

```
wordVal SWORD -5000
```

```
.code
```

```
mov ax, wordVal
```

```
cwd
```

```
mov bx, +256
```

```
idiv bx
```

	Answers	Registers (EAX/EBX/ECX/EDX)
Quotient	Ffed	AX
Remainder	Ff78	DX

Question 04**(10 Marks)**

Write a **recursive** assembly language program to calculate the n-th term of the **Pell series**. The Pell series is defined as follows:

$$P(n) = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ 2 \cdot P(n-1) + P(n-2) & \text{if } n > 1 \end{cases}$$

An example of the series till n = 4 is 0, 1, 2, 5, 12.

```

.386

.model flat, stdcall

.stack 4096

include Irvine32.inc

.data

n DWORD 6                ; Desired term (n) in the Pell series

.code

main PROC

    mov eax, n            ; Load the term number (n) into eax
    push eax              ; Push n as a parameter for Pell(n)
    call Pell             ; Call the Pell function
    add esp, 4            ; Clean up the stack

    ; Result is now in eax, exit
    exit

main ENDP

Pell PROC

    push ebp              ; Save base pointer
    mov ebp, esp          ; Establish stack frame
    sub esp, 4            ; Allocate local variable

    mov eax, [ebp + 8]    ; Get the input parameter (n)

```

```

    cmp eax, 0          ; Compare n with 0
    je base_case_0      ; If n == 0, go to base case 0

    cmp eax, 1          ; Compare n with 1
    je base_case_1      ; If n == 1, go to base case 1

    ; Recursive case: 2 * Pell(n-1) + Pell(n-2)

    push eax            ; Save n
    dec eax             ; n = n - 1
    push eax            ; Push n-1
    call Pell           ; Compute Pell(n-1)
    add esp, 4          ; Clean up the stack
    mov ebx, eax        ; Save Pell(n-1) in ebx

    pop eax             ; Restore n
    dec eax             ; n = n - 2
    push eax            ; Push n-2
    call Pell           ; Compute Pell(n-2)
    add esp, 4          ; Clean up the stack

    lea eax, [ebx + eax * 2] ; eax = 2 * Pell(n-1) + Pell(n-2)
    jmp end_pell        ; End of the function

base_case_0:
    mov eax, 0          ; Pell(0) = 0
    jmp end_pell        ; End of the function

```

```
base_case_1:
    mov eax, 1          ; Pell(1) = 1
    jmp end_pell        ; End of the function

end_pell:
    mov esp, ebp        ; Restore stack pointer
    pop ebp             ; Restore base pointer
    ret                 ; Return to the caller

Pell ENDP

END main
```